

수업 05 · 전체 과정 10/20일차

AI 코드에이전트

2026-10-20 (화) · 10:00~17:00

50분 수업 · 10분 휴식 · 중식 11:50~13:00

AI에 맡길 일을 구체적으로 적고, 제안된 코드의 변경과 결과를 확인합니다. 제공 응답을 비교하며 요청과 검토를 연습합니다.

교안 읽기	직접 해보기	결과 확인하기
개념과 완성 예제를 먼저 봅니다.	필수 활동을 열어 작성·수정합니다.	예상과 실제를 비교하고 설명합니다.

처음 배우는 사람을 위한 수업

파일과 폴더, 저장과 실행부터 설명합니다. 설치·계정 문제가 있으면 제공 예제의 수동 추적이나 강사 시연으로 같은 개념을 확인하고, 실제 실행 여부를 구분해 기록합니다.

웹 강의실 열기 · 화면의 링크와 내려받기 자료를 함께 사용하세요.

PDF 제작일 2026-10-05 · 원본 웹 내용 + 50분 교시 운영·설명 보완

이 교안의 순서

오늘 준비할 것	3
오늘의 6교시	4
1교시 · 작업을 맡기는 방법	5
2교시 · 근거가 있는 요청서	11
3교시 · 변경된 코드 읽기	18
4교시 · 예상과 실제 결과	24
5교시 · 오류 재현과 수정	31
6교시 · 새 요청과 인수인계	37
용어와 코드 기호 찾아보기	44
오늘 배운 내용 확인하기	45
파일과 다음 수업	47

오늘 준비할 것

처음 시작하기 · AI 코드에이전트

1. ZIP을 일반 폴더에 풀고 examples/course의 data·work·code 위치를 유지합니다. 배포 ZIP은 course 폴더부터 열 수 있습니다.
2. [교안](#)에서 개념을 읽고 [실습지](#)의 A01부터 진행합니다.
3. work 파일만 수정하고 원본은 보존합니다. 같은 파일을 저장·다시 열어 확인합니다.
4. 개인 실행을 못 했으면 예상과 강사 실행 결과를 대조하고 확인 방법을 정확히 적습니다.

실행 준비

브라우저와 텍스트 편집기가 기본입니다. Python 명령의 현재 폴더는 압축을 푼 course입니다. Windows에서 python 대신 py만 있으면 강사의 확인 아래 py -3로 대체합니다.

```
python code/check_cases.py code/baseline.py
python code/check_cases.py code/proposal_a.py
python code/check_cases.py code/reviewed.py
python code/check_cases.py work/classifier.py
```

실행하지 못하면 파일 경로·명령·오류 문구를 기록해 도움을 요청합니다. 코드 상자를 보는 것은 Python 실행이 아닙니다. 실제 AI 계정·모델·유료 API를 호출하지 않습니다. 제공 응답은 강사가 만든 비교 자료입니다. 합성 분류 규칙이며 실제 센서 안전 판단을 검증하지 않았습니다.

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 자기 말로 답한 뒤 풀이와 비교하세요.

요청과 정답

질문: AI가 실행할 수 있는 코드를 내놓으면 업무 요구를 만족하나요?

풀이: 실행 성공과 요구 충족은 다릅니다. 구체적인 입력, 규칙, 기대 결과로 비교해야 합니다.

근거

질문: AI가 모두 테스트했다고 말하면 확인을 끝내도 될까요?

풀이: 실제 실행 명령, 입력, 출력과 실패 사례를 확인합니다. 수행하지 않은 검증은 별도로 남깁니다.

범위

질문: 한 함수 수정을 맡겼는데 전체 파일이 바뀌면 무엇을 보나요?

풀이: 변경 차이를 읽고 요청 범위와 비교합니다. 필요 없는 변경을 분리한 뒤 정상·경계·누락 입력을 확인합니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 독립 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

오늘의 6교시

교시·시간	학습 초점	필수 활동
1교시 10:00~10:50	작업을 맡길 범위와 자료	A01, A02, A03
2교시 11:00~11:50	검증 가능한 요청	A04, A05, A06
3교시 13:00~13:50	응답 주장과 실제 변경	A07, A08, A09
4교시 14:00~14:50	작은 수정과 재검사	A10, A11, A12
5교시 15:00~15:50	오류 재현과 재요청	A13, A14, A15
6교시 16:00~16:50	새 기준과 인계	A16, A17, A18

설명·시연·함께 수행·개별 수행·폴이를 합쳐 교시당 50분입니다. 추가 활동까지 모두 같은 시간에 끝내라는 뜻은 아닙니다. 마지막 10분은 휴식·종료 준비로 배치합니다. 실제 학습자 소요시간은 수업에서 측정해 조정합니다.

이전 Python·Git 기초를 짧게 복습하며 준비된 작은 예제를 읽고 수정합니다. **최신 운영은 50분 × 6교시, 순수 수업 300분**입니다. 설명 보충은 필수 개념이며 독립 과제는 추가 복습입니다. 추가 문제는 수업 후 복습에 사용합니다.

실제 AI 계정·모델·유료 API를 호출하지 않습니다. 제공 응답은 강사가 만든 비교 자료입니다. 합성 분류 규칙이며 실제 센서 안전 판단을 검증하지 않았습니다.

전체 코드와 모든 줄의 뜻·실행 방법은 [코드 해설](#)에 있습니다. 작성 소스와 생성된 번들·잠금 파일을 구분합니다.

1교시 · 작업을 맡기는 방법

10:00~10:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 폴이·확인 7분

이번 교시에서 확인할 것. 작업을 맡길 범위와 자료

먼저 할 활동: A01, A02, A03

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A01 · 에이전트와 사람의 역할

사례로 이해하기 · 실행하는 AI와 책임을 갖는 사람

실무 상황

센서 안내 프로그램을 고치려는 직원이 AI에게 “온도 분류를 개선해 줘”라고 보냅니다. AI는 파일 이름과 기준을 추측하고 새 코드를 제안할 수 있습니다. 직원이 기대한 것은 경계 오류 하나의 수정인데 프로그램 전체가 바뀌면 검토가 어려워집니다. 먼저 맡길 범위와 사람이 결정할 규칙을 구별합니다.

쉬운 폴이

대화형 AI는 문장이나 코드를 답할 수 있습니다. 코드에이전트는 허용된 도구를 이용하여 파일을 읽고 고치거나 실행까지 이어갈 수 있습니다. 가능한 능력과 이번 작업에서 허용한 행동은 다릅니다. 프롬프트는 요청 글이고 컨텍스트는 그 요청을 이해하는 데 필요한 자료입니다. 사람은 업무 기준, 공유할 자료, 적용 승인, 실제 확인 범위를 결정합니다. 도구가 읽지 못한 숨은 규칙까지 안다고 추정하지 않습니다.

완성 예와 폴이

요청의 목적을 “28 이상을 주의로 분류”로 적습니다. 대상은 work/classifier.py의 classify 함수이며 입력은 가상 온도, 출력은 정상·주의·미측정·입력 오류입니다. 관련 자료는 현재 함수와 cases.json, 실패 출력 마지막 부분입니다. 읽기와 수정은 이 연습 복사본에만 허용하고 외부 게시와 계정 접근은 허용하지 않습니다. 실제 토큰 대신 **DEMO_TOKEN**이라는 표시를 씁니다. 확인 근거는 변경 비교와 같은 검사 입력의 실행 결과입니다. 제공 AI 응답은 수업용 작성 자료이며 실제 모델 호출 증거가 아닙니다.

독립 추가 복습 과제

파일 세 개, 고객 명단, 접속 암호, 가상 실패 로그 가운데 온도 경계 수정에 필요한 자료를 선택하세요. 이어서 “목적 / 바꿀 곳 / 유지할 것 / 금지할 행동 / 확인 근거” 다섯 줄 요청서를 작성하세요.

답과 대표 오답

현재 함수·검사 입력·가상 실패 로그를 선택하고 고객 명단과 암호는 제외합니다. “자료를 전부 보내면 정확해진다”는 답은 관련성과 공유 범위를 검토하지 않았습니다. “AI가 썼으니 실행도 했다” 역시 생성과 실행의 차이를 놓칩니다.

확인 질문과 해설

① 에이전트 이름이 능력을 보장하나요? 실제 연결 도구와 권한을 확인해야 합니다. ② 비밀이 필요한 상황은 어떻게 하나요? 그대로 공유하지 말고 담당 절차와 대체 데이터를 확인합니다. ③ 완료 판단은 누구의 일인가요? 요구와 증거를 대조하는 사람의 판단이 필요합니다.

뜻과 비유. 코드에이전트는 요청을 해석하고 파일과 실행 도구를 사용해 작업을 이어 가는 AI 도구입니다. 대화 답변 한 번과 파일 변경·실행이 연결된 작업 흐름을 구분합니다.

왜 필요한가. 동료에게 일을 맡겨도 완료 기준은 함께 정합니다. 코드가 생겼다는 사실과 업무 요구가 충족됐다는 사실은 다릅니다.

예를 따라 읽기. 센서 온도 28을 주의로 분류해 달라는 요청은 입력·기준·출력·확인 방법을 포함할 수 있습니다.

햇갈리는 반례. “알아서 좋게 고쳐”는 유지해야 할 함수 이름이나 28 °C 경계를 알려 주지 않습니다.

직접 적용. 요청서에서 목적을 한 문장으로 찾기 → AI가 제안할 일과 사람이 판단할 일 나누기 → 완료라고 부를 증거 두 가지 적기 → 짝에게 도구와 검토자의 역할 설명하기

설명할 질문. 코드가 생긴 것만으로 완료가 아닌 이유는 무엇인가요?

막혔을 때. 도구 이름과 능력을 혼동하면 읽기·변경·실행 중 실제로 허용한 행동을 따로 적습니다.

입력·답변·확인 순서는 [실습지의 A01](#)와 [활동 원문](#)에 있습니다.

A01 · 따라 하기와 작성칸

A01 · 에이전트와 사람의 역할

목적

동료에게 일을 맡겨도 완료 기준은 함께 정합니다. 코드가 생겼다는 사실과 업무 요구가 충족됐다는 사실은 다릅니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [request.md](#), [A01_input.md](#)

작업: [A01_notes.md](#)

실제 입력

센서 온도 28을 주의로 분류해 달라는 요청은 입력·기준·출력·확인 방법을 포함할 수 있습니다.

단계

- 1. 요청서에서 목적을 한 문장으로 찾기
- 2. AI가 제안할 일과 사람이 판단할 일 나누기
- 3. 완료라고 부를 증거 두 가지 적기
- 4. 짝에게 도구와 검토자의 역할 설명하기

예상 결과

목적·역할·완료 근거가 있는 작업 설명

확인 방법

코드가 생긴 것만으로 완료가 아닌 이유는 무엇인가요?

오류 해결

도구 이름과 능력을 혼동하면 읽기·변경·실행 중 실제로 허용한 행동을 따로 적습니다.

힌트

코드에이전트는 요청을 해석하고 파일과 실행 도구를 사용해 작업을 이어 가는 AI 도구입니다. 대화 답변 한 번과 파일 변경·실행이 연결된 작업 흐름을 구분합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 코드가 생긴 것만으로 완료가 아닌 이유는 무엇인가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
실제 결과·근거: _____
다음에 확인할 것: _____

A02 · 작은 작업과 수정 범위

- 뜻과 비유.** 작업 범위는 어떤 파일과 동작을 바꿀지 정한 경계입니다. 전체 프로그램을 한 번에 고치기보다 한 함수의 분류 규칙처럼 확인 가능한 단위로 나눕니다.
- 왜 필요한가.** 범위를 알면 변경 비교가 짧아지고 잘못 바꾼 항목을 되돌리기 쉽습니다. 10/16의 변경 비교가 여기서 업무 검토 근거가 됩니다.
- 예를 따라 읽기.** 오늘은 classifier.py의 classify 동작만 바꾸고 함수 이름과 threshold =28을 유지합니다.
- 헛갈리는 반례.** 결과를 맞추려고 cases.json 기대 값을 변경하면 검사기준을 바꾼 것이므로 요청을 해결한 증거가 아닙니다.
- 직접 적용.** 바꿀 파일 하나 적기 → 유지할 이름·기준·입출력 찾기 → 요청 밖 변경 예 두 가지 표시하기 → 작업 요청서에 범위 적기
- 설명할 질문.** 테스트 기대 값을 바꾸면 왜 문제가 숨을 수 있나요?
- 막혔을 때.** 파일을 못 찾으면 압축을 푼 course 폴더의 code·data·work 위치부터 확인합니다.
- 입력·답안·확인 순서는 [실습지의 A02](#)와 [활동 원문](#)에 있습니다.

A02 · 따라 하기와 작성칸

A02 · 작은 작업과 수정 범위

목적

범위를 알면 변경 비교가 짧아지고 잘못 바꾼 항목을 되돌리기 쉽습니다. 10/16의 변경 비교가 여기서 업무 검토 근거가 됩니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답안과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [request.md](#), [task.md](#), [A02_input.md](#)

작업: [A02_notes.md](#), [task.md](#)

실제 입력

오늘은 classifier.py의 classify 동작만 바꾸고 함수 이름과 threshold =28을 유지합니다.

단계

1. 바꿀 파일 하나 적기
2. 유지할 이름·기준·입출력 찾기
3. 요청 밖 변경 예 두 가지 표시하기
4. 작업 요청서에 범위 적기

예상 결과

변경 가능·유지 범위가 구분된 요청서

확인 방법

테스트 기대 값을 바꾸면 왜 문제가 숨을 수 있나요?

오류 해결

파일을 못 찾으면 압축을 푼 course 폴더의 code·data·work 위치부터 확인합니다.

힌트

작업 범위는 어떤 파일과 동작을 바꿀지 정한 경계입니다. 전체 프로그램을 한 번에 고치기보다 한 함수의 분류 규칙처럼 확인 가능한 단위로 나눕니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 테스트 기대 값을 바꾸면 왜 문제가 숨을 수 있나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A03 · 컨텍스트와 관련 자료

뜻과 비유. 컨 텍스트는 이번 일을 이해하는 데 필요한 설명과 자료입니다. 파일 전체를 많이 넣는 것보다 어떤 입력이 어떤 문제를 만드는지 정확히 전달하는 것이 중요합니다.

왜 필요한가. 도구가 보지 못한 파일이나 숨은 업무 규칙을 저절로 안다고 가정하면 엉뚱한 수정이 나올 수 있습니다.

예를 따라 읽기. 원본 함수·가상 입력·기대 결과·실제 오류 마지막 줄은 문제를 재현하는 데 도움이 됩니다.

햇갈리는 반례. 관계없는 오래된 프로젝트와 실제 계정 비밀번호는 오늘의 분류 규칙을 설명하는 데 필요하지 않습니다.

직접 적용. 여섯 자료 카드 읽기 → 관련·무관·공유하지 않을 자료로 분류하기 → 선택한 자료의 필요성을 한 줄씩 쓰기 → 빠진 맥락 하나를 질문으로 남기기

설명할 질문. 자료가 많으면 항상 더 좋은 요청이 되나요?

막혔을 때. 출처와 사용 목적을 구분하지 못하면 각 자료가 어떤 질문에 답하는지 적어 봅니다.

입력·답칸·확인 순서는 [실습지의 A03](#)와 [활동 원문](#)에 있습니다.

A03 · 따라 하기와 작성칸

A03 · 컨텍스트와 관련 자료

목적

도구가 보지 못한 파일이나 숨은 업무 규칙을 저절로 안다고 가정하면 엉뚱한 수정이 나올 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [context_cards.md](#), [A03_input.md](#)

작업: [A03_notes.md](#)

실제 입력

원본 함수·가상 입력·기대 결과·실제 오류 마지막 줄은 문제를 재현하는 데 도움이 됩니다.

단계

1. 여섯 자료 카드 읽기
2. 관련·무관·공유하지 않을 자료로 분류하기
3. 선택한 자료의 필요성을 한 줄씩 쓰기
4. 빠진 맥락 하나를 질문으로 남기기

예상 결과

최소 관련 자료 목록과 선택 이유

확인 방법

자료가 많으면 항상 더 좋은 요청이 되나요?

오류 해결

출처와 사용 목적을 구분하지 못하면 각 자료가 어떤 질문에 답하는지 적어 봅니다.

힌트

컨 텍스트는 이번 일을 이해하는 데 필요한 설명과 자료입니다. 파일 전체를 많이 넣는 것보다 어떤 입력이 어떤 문제를 만드는지 정확히 전달하는 것이 중요합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 자료가 많으면 항상 더 좋은 요청이 되나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
실제 결과·근거: _____
다음에 확인할 것: _____

2교시 · 근거가 있는 요청서

11:00~11:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 검증 가능한 요청

먼저 할 활동: A04, A05, A06

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A04 · 입력과 출력 계약

사례로 이해하기 · 요청을 검사 가능한 약속으로 바꾸기

실무 상황

동료가 “높은 온도를 주의로 표시”라고만 설명합니다. 28은 높은 값인가요? 값이 없으면 무엇을 표시하나요? 기계가 이해할 조건을 쓰기 전에 사람끼리 애매한 표현을 정확한 입출력 약속으로 바뀌어야 합니다. 코드 한 줄의 차이가 사용자 안내를 바꾸기 때문입니다.

쉬운 풀이

계약은 함수가 받을 값과 돌려줄 결과의 약속입니다. 정상 사례는 흔한 입력, 경계 사례는 규칙이 바뀌는 근처 입력, 오류 사례는 허용하지 않은 입력입니다. 온도 0은 측정값이고 None은 값이 없다는 표시입니다. 글자 "28"과 숫자 28은 화면에 비슷해 보여도 자료형이 다릅니다. 오늘 계약은 문자 숫자를 자동 변환하지 않고 입력 오류로 처리합니다. 업무 규칙이 달라지면 별도 승인이 필요합니다.

완성 예와 풀이

함수 이름 classify와 기본 기준 28을 유지합니다. 유효한 수는 0~50이며 28 이상은 주의, 미만은 정상입니다. None은 미측정, 문자·불리언·범위 밖 값은 입력 오류입니다. `classify(27.9)`→정상, `classify(28)`→주의, `classify(28.1)`→주의, `classify(None)`→미측정으로 표를 작성합니다. `classify(0)`→정상을 별도 행으로 두면 None을 0으로 바꾼 후보의 문제를 찾을 수 있습니다. 기준을 27로 내리면 27.9의 약속이 깨집니다. 검사 파일의 기대 값을 바꾸어서 후보를 통과시키면 요구 검증이 되지 않습니다.

독립 추가 복습 과제

냉장 보관 가상 규칙은 "5 초과이면 점검, 5 이하는 정상"입니다. 4.9·5·5.1의 기대를 직접 쓰고 온도 기준의 "이상" 규칙과 어떤 비교 기호가 다른지 설명하세요. 누락 값의 처리 약속도 한 문장 작성하세요.

답과 대표 오답

4.9와 5는 정상, 5.1은 점검이며 초과는 >입니다. "5도 점검"은 초과와 이상을 혼동했습니다. 누락은 측정하지 못했음을 별도로 표시해야 하며 정상으로 채우는 답은 유효 측정과 누락을 합칩니다.

확인 질문과 해설

① 함수 이름을 유지하는 이유는 무엇인가요? 기존 호출자가 같은 이름을 사용합니다. ② 경계 세 값을 두는 이유는 무엇인가요? 기준과 비교 기호의 오류를 구별합니다. ③ 출력 표만 맞으면 충분한가요? 실제 대상 함수의 실행과 유지 조건도 확인합니다.

뜻과 비유. 입출력 계약은 어떤 값을 받아 어떤 형태로 답할지 정한 약속입니다. 함수의 내부가 달라져도 호출하는 쪽이 기대하는 이름과 결과는 유지해야 합니다.

왜 필요한가. 함수 이름을 바꾸거나 새 숫자 코드를 반환하면 기존 호출자는 결과를 이해하지 못할 수 있습니다.

예를 따라 읽기. `classify(28)`는 주의, `classify(None)`는 미측정으로 답하도록 요청에 씁니다.

헛갈리는 반례. 미측정을 정상으로 부르면 측정하지 못한 상황이 정상 온도로 보입니다.

직접 적용. 요청서에서 허용 입력 추출하기 → 네 반환 문자열을 표로 쓰기 → 함수 호출 하나의 입력·출력 연결하기 → task.md에 계약 작성하기

설명할 질문. 함수 이름과 반환 문자열을 유지하는 이유는 무엇인가요?

막혔을 때. None과 문자열 "None"이 헛갈리면 10/8의 값 없음과 글자 차이를 다시 비교합니다.

입력·답칸·확인 순서는 [실습지의 A04](#)와 [활동 원문](#)에 있습니다.

A04 · 따라 하기와 작성칸

A04 · 입력과 출력 계약

목적

함수 이름을 바꾸거나 새 숫자 코드를 반환하면 기존 호출 자는 결과를 이해하지 못할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [request.md](#), [baseline.py](#), [task.md](#), [A04_input.md](#)

작업: [A04_notes.md](#), [task.md](#)

실제 입력

classify(28)는 주의, classify(None)는 미측정으로 답하도록 요청에 씁니다.

단계

1. 요청서에서 허용 입력 추출하기
2. 네 반환 문자열을 표로 쓰기
3. 함수 호출 하나의 입력·출력 연결하기
4. task.md에 계약 작성하기

예상 결과

입력 종류와 네 출력 상태를 명시한 계약

확인 방법

함수 이름과 반환 문자열을 유지하는 이유는 무엇인가요?

오류 해결

None과 문자열 "None"이 헛갈리면 10/8의 값 없음과 글자 차이를 다시 비교합니다.

힌트

입출력 계약은 어떤 값을 받아 어떤 형태로 답할지 정한 약속입니다. 함수의 내부가 달라져도 호출하는 쪽이 기대하는 이름과 결과는 유지해야합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 함수 이름과 반환 문자열을 유지하는 이유는 무엇인가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A05 · 정상·경계·오류 사례

뜻과 비유. 검사 사례는 입력과 기대 결과를 한 쌍으로 적은 것입니다. 경계 사례는 규칙이 달라지는 바로 앞·같은 값·바로 뒤를 비교합니다.

왜 필요한가. 평범한 22.5 하나가 통과해도 28에서 비교 기호가 틀린 문제는 드러나지 않습니다.

예를 따라 읽기. 27.9는 정상, 28 과 28.1은 주의입니다. 0은 유효하며 None은 미측정입니다.

헛갈리는 반례. 글자 “뜨거움”을 0으로 바꿔 처리하면 오류 입력이 정상 기록으로 둔갑합니다.

직접 적용. cases.json 아홉 입력 읽기 → 실행 전에 기대 상태 써 보기 → 경계 세 사례를 묶기 → 결과가 틀릴 때의 업무 영향을 설명하기

설명할 질문. 왜 28만 확인하지 않고 27.9와 28.1도 보나요?

막혔을 때. 숫자와 불리 언을 혼동하면 type(True)가 bool임을 확인하고 온도 계약과 비교합니다.

입력·답칸·확인 순서는 [실습지의 A05](#)와 [활동 원문](#)에 있습니다.

A05 · 따라 하기와 작성칸

A05 · 정상·경계·오류 사례

목적

평범한 22.5 하나가 통과해도 28에서 비교 기호가 틀린 문제는 드러나지 않습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [cases.json](#), [request.md](#), [A05_input.md](#)

작업: [A05_notes.md](#)

실제 입력

27.9는 정상, 28 과 28.1은 주의입니다. 0은 유효하며 None은 미측정입니다.

단계

1. cases.json 아홉 입력 읽기
2. 실행 전에 기대 상태 써 보기
3. 경계 세 사례를 묶기
4. 결과가 틀릴 때의 업무 영향을 설명하기

예상 결과

아홉 사례의 예상과 경계 근거

확인 방법

왜 28만 확인하지 않고 27.9와 28.1도 보나요?

오류 해결

숫자와 불리 연을 혼동하면 type(True)가 bool임을 확인하고 온도 계약과 비교합니다.

힌트

검사 사례는 입력과 기대 결과를 한 쌍으로 적은 것입니다. 경계 사례는 규칙이 달라지는 바로 앞·같은 값·바로 뒤를 비교합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 왜 28만 확인하지 않고 27.9와 28.1도 보나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A06 · 검증 가능한 프롬프트

뜻과 비유. 프롬프트는 도구에게 전달하는 요청 문장입니다. 목표·입력·범위·유지 조건·확인 사례를 짧게 연결하면 모호한 칭찬 대신 확인 가능한 결과를 요구할 수 있습니다.

왜 필요한가. 검사 방법을 먼저 적으면 도구가 스스로 완료라고 말해도 무엇이 확인됐는지 비교할 수 있습니다.

예를 따라 읽기. “28 이상은 주의로 고치고 None은 미측정으로 남기며 아홉 사례를 실행해 결과를 보고해 주세요.”

헛갈리는 반례. “완벽한 전문가 코드로 만들어”는 어떤 결과가 맞는지 판단할 기준이 없습니다.

직접 적용. work/task.md의 여섯 칸 채우기 → 모호한 형용사 한 개를 구체 조건으로 고치기 → 짝이 요청만 보고 예상 결과 말하기 → 해석이 갈린 문장 다시 쓰기

설명할 질문. 좋은 요청에 실행 전 기대 결과가 들어가는 이유는 무엇인가요?

막혔을 때. 요청이 길어지면 목적과 유지 조건을 분리하고 이번에 필요 없는 기능은 후속 요청으로 둡니다.

입력·답칸·확인 순서는 [실습지의 A06](#)과 [활동 원문](#)에 있습니다.

A06 · 따라 하기와 작성칸

A06 · 검증 가능한 프롬프트

목적

검사 방법을 먼저 적으면 도구가 스스로 완료라고 말해도 무엇이 확인됐는지 비교할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [task.md](#), [request.md](#), [A06_input.md](#)

작업: [A06_notes.md](#), [task.md](#)

실제 입력

“28 이상은 주의로 고치고 None은 미측정으로 남기며 아홉 사례를 실행해 결과를 보고해 주세요.”

단계

1. work/task.md의 여섯 칸 채우기
2. 모호한 형용사 한 개를 구체 조건으로 고치기
3. 짝이 요청만 보고 예상 결과 말하기
4. 해석이 갈린 문장 다시 쓰기

예상 결과

입력·제약·검사 가능한 작업 요청

확인 방법

좋은 요청에 실행 전 기대 결과가 들어가는 이유는 무엇인가요?

오류 해결

요청이 길어지면 목적과 유지 조건을 분리하고 이번에 필요 없는 기능은 후속 요청으로 둡니다.

힌트

프롬프트는 도구에게 전달하는 요청 문장입니다. 목표·입력·범위·유지 조건·확인 사례를 짧게 연결하면 모호한 칭찬 대신 확인 가능한 결과를 요구할 수 있습니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 좋은 요청에 실행 전 기대 결과가 들어가는 이유는 무엇인가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

3교시 · 변경된 코드 읽기

13:00~13:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 응답 주장과 실제 변경

먼저 할 활동: A07, A08, A09

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A07 · 변경 비교의 기호 읽기

사례로 이해하기 · 그럴듯한 완료 문장에서 근거 찾기

실무 상황

제공 응답 A는 “모든 문제를 해결했습니다”라고 말합니다. 하지만 코드에서 기본 기준을 27로 낮추고 None을 0으로 바꿨습니다. 친절하고 자신 있는 문장은 업무 규칙을 충족했다는 증거가 아닙니다. 검토자는 주장, 실제 변경, 실행 기록을 각각 찾아 연결해야 합니다.

쉬운 풀이

변경 비교는 이전과 이후의 차이를 보여 줍니다. 삭제 표시 -와 추가 표시 +는 파일 내용에 넣는 기호가 아닙니다. 주장에는 “고쳤다”, “검사했다”, “운영 가능하다”처럼 서로 다른 범위가 있습니다. 첫 주장은 코드 차이로, 둘째는 대상과 명령과 출력으로, 셋째는 실제 환경 증거로 확인합니다. 증거가 없으면 거짓으로 단정하기보다 확인하지 못했다고 적고 다음 검사 질문을 만듭니다.

완성 예와 풀이

code/proposal_a.py를 읽으면 `threshold=27`과 None을 0으로 바꾸는 줄이 보입니다. 요청의 유지 조건은 `threshold=28`이고 None은 미측정입니다. 먼저 두 차이를 표시한 뒤 같은 code/check_cases.py로 실행합니다. 응답의 말과 실제 함수 동작은 별도로 기록합니다. 실행 경로가 proposal_a.py인지도 확인합니다. work/classifier.py를 고쳤는데 baseline.py를 실행했다면 고친 내용에 대한 증거가 아닙니다. 복구할 때는 원본 baseline.py를 보존한 채 작업 복사본만 다시 가져옵니다.

독립 추가 복습 과제

“수정 완료, 오류 없음, 실제 장비에서도 안정적”이라는 세 주장 옆에 필요한 증거를 한 종류씩 적으세요. 주어진 자료가 코드 비교와 로컬 가상 검사 출력뿐일 때 어느 문장까지 보고할 수 있는지도 결정하세요.

답과 대표 오답

변경 완료에는 diff, 오류 검사는 사례별 실행 결과, 장비 안정성에는 실제 장비와 운영 조건의 관찰이 필요합니다. 현재는 가상 검사 범위만 보고하고 장비 부분은 미확인으로 남깁니다. "AI의 자신감이 높아서 승인"은 검증 근거가 없습니다. "검사 결과가 없으니 코드는 반드시 틀렸다"도 증거 부족과 실패를 혼동합니다.

확인 질문과 해설

① 코드 차이만으로 실행 성공을 아나요? 실행 증거가 필요합니다. ② 통과 화면에 대상 경로가 필요한 이유는 무엇인가요? 어느 복사본을 확인했는지 식별합니다. ③ 불확실성을 어떻게 쓰나요? 관찰 사실과 아직 확인하지 않은 조건을 나누어 구체적으로 적습니다.

뜻과 비유. diff는 전후 코드에서 빠진 줄과 새 줄을 보여 주는 변경 비교입니다. -는 이전 줄, +는 새 줄이며 코드의 빠기·더하기 연산과 다릅니다.

왜 필요한가. 전체 파일을 외우지 않아도 요청이 어느 변경에 반영됐는지 확인할 수 있습니다.

예를 따라 읽기. >를 >=로 바꾼 줄은 경계 28을 포함하도록 동작을 바꿉니다.

헛갈리는 반례. 주석만 "28 이상"으로 바뀌고 실제 비교가 >이면 동작은 그대로입니다.

직접 적용. changes.diff에서 파일 이름 찾기 → 미측정·검사·경계 변경 줄 묶기 → 요청한 항목과 변경 한 곳 연결하기 → 문구 변경과 동작 변경 구분하기

설명할 질문. diff의 +와 Python의 +는 어떤 차이가 있나요?

막혔을 때. diff의 +를 원본 코드에 복사했다면 실제 소스 파일의 완성 줄을 참고해 제거합니다.

입력·답칸·확인 순서는 [실습지의 A07](#)과 [활동 원문](#)에 있습니다.

A07 · 따라 하기와 작성칸

A07 · 변경 비교의 기호 읽기

목적

전체 파일을 외우지 않아도 요청이 어느 변경에 반영됐는지 확인할 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [changes.diff](#), [baseline.py](#), [reviewed.py](#), [A07_input.md](#)

작업: [A07_notes.md](#)

실제 입력

를 >=로 바꾼 줄은 경계 28을 포함하도록 동작을 바꿉니다.

단계

- 1. changes.diff에서 파일 이름 찾기
- 2. 미측정·검사·경계 변경 줄 묶기
- 3. 요청한 항목과 변경 한 곳 연결하기
- 4. 문구 변경과 동작 변경 구분하기

예상 결과

요청과 실제 변경 줄의 대응 표

확인 방법

diff의 +와 Python의 +는 어떤 차이가 있나요?

오류 해결

diff의 +를 원본 코드에 복사했다면 실제 소스 파일의 완성 줄을 참고해 제거합니다.

힌트

diff는 전후 코드에서 빠진 줄과 새 줄을 보여 주는 변경 비교입니다. -는 이전 줄, +는 새 줄이며 코드의 빼기·더하기 연산과 다릅니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: diff의 +와 Python의 +는 어떤 차이가 있나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A08 · 설득력 있는 응답 검토

뜻과 비유. AI 응답은 설명이며 실행 증거 그 자체가 아닙니다. 코드를 바꿨다는 주장, 검사를 했다는 주장, 실제 결과를 각각 확인합니다.

왜 필요한가. 자신 있는 문장 때문에 범위 밖 수정이나 누락된 검사를 지나칠 수 있습니다.

예를 따라 읽기. 응답 A는 기준을 27로 바꾸고 None을 0으로 채웁니다. 요청의 28 유지와 미측정 보존에 어긋납니다.

헛갈리는 반례. “모든 문제 해결”이라는 문장이 있어도 어떤 입력으로 확인했는지 없으면 결론을 보류합니다.

직접 적용. 응답 A와 코드 나란히 읽기 → 기준 변경과 빈 값 처리 표시하기 → 요구 위반 두 개를 근거로 쓰기 → 승인 전 추가 확인 질문 작성하기

설명할 질문. 친절하고 자신 있는 응답도 왜 다시 확인하나요?

막혔을 때. 응답의 어조보다 실제 바뀐 값과 반환을 원문 요청에 대조합니다.

입력·답안 확인 순서는 [실습지의 A08](#)와 [활동 원문](#)에 있습니다.

A08 · 따라 하기와 작성칸

A08 · 설득력 있는 응답 검토

목적

자신 있는 문장 때문에 범위 밖 수정이나 누락된 검사를 지나칠 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [agent_response_a.md](#), [proposal_a.py](#), [request.md](#), [A08_input.md](#)

작업: [A08_notes.md](#)

실제 입력

응답 A는 기준을 27로 바꾸고 None을 0으로 채웁니다. 요청의 28 유지와 미측정 보존에 어긋납니다.

단계

1. 응답 A와 코드 나란히 읽기
2. 기준 변경과 빈 값 처리 표시하기
3. 요구 위반 두 개를 근거로 쓰기
4. 승인 전 추가 확인 질문 작성하기

예상 결과

응답의 주장과 코드 근거를 분리한 검토

확인 방법

친절하고 자신 있는 응답도 왜 다시 확인하나요?

오류 해결

응답의 어조보다 실제 바뀐 값과 반환을 원문 요청에 대조합니다.

힌트

AI 응답은 설명이며 실행 증거 그 자체가 아닙니다. 코드를 바꿨다는 주장, 검사를 했다는 주장, 실제 결과를 각각 확인합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 친절하고 자신 있는 응답도 왜 다시 확인하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A09 · 작업 복사본과 되돌림

뜻과 비유. 작업 복사본은 원본을 남기고 수정 결과를 비교하기 위한 파일입니다. 저장한 파일과 실행한 파일의 경로가 같아야 변경의 효과를 볼 수 있습니다.

왜 필요한가. 같은 이름의 파일을 다른 폴더에서 실행하면 고쳤는데도 이전 결과가 나오는 것처럼 보입니다.

예를 따라 읽기. work/classifier.py는 원본과 같은 시작 코드입니다. code/baseline.py는 비교용으로 유지합니다.

헛갈리는 반례. 원본까지 덮어쓰고 이 전 동작을 기억만 하면 변경 근거와 되돌릴 기준을 잃습니다.

직접 적용. 세 폴더 역할 적기 → work/classifier.py를 열어 원본과 대조하기 → 실행할 정확 경로 기록하기 → 수정전 사본과 복구 방법 적기

설명할 질문. 파일 이름이 같은데 다른 결과가 나올 수 있는 이유는 무엇인가요?

막혔을 때. 파일 확장자 숨김이면 파일 속성을 확인하고 ZIP 목록 안에서 편집하지 않습니다.

입력·답칸·확인 순서는 [실습지의 A09](#)와 [활동 원문](#)에 있습니다.

A09 · 따라 하기와 작성칸

A09 · 작업 복사본과 되돌림

목적

같은 이름의 파일을 다른 폴더에서 실행하면 고쳤는데도 이전 결과가 나오는 것처럼 보입니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [baseline.py](#), [classifier.py](#), [A09_input.md](#)

작업: [A09_notes.md](#), [classifier.py](#)

실제 입력

work/classifier.py는 원본과 같은 시작 코드입니다. code/baseline.py는 비교용으로 유지합니다.

단계

1. 세 폴더 역할 적기
2. work/classifier.py를 열어 원본과 대조하기
3. 실행할 정확 경로 기록하기
4. 수정전 사본과 복구 방법 적기

예상 결과

원본·작업본·실행 경로와 복구 기준

확인 방법

파일 이름이 같은데 다른 결과가 나올 수 있는 이유는 무엇인가요?

오류 해결

파일 확장자 숨김이면 파일 속성을 확인하고 ZIP 목록 안에서 편집하지 않습니다.

힌트

작업 복사본은 원본을 남기고 수정 결과를 비교하기 위한 파일입니다. 저장한 파일과 실행한 파일의 경로가 같아야 변경의 효과를 볼 수 있습니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 파일 이름이 같은데 다른 결과가 나올 수 있는 이유는 무엇인가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

4교시 · 예상과 실제 결과

14:00~14:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 작은 수정과 재검사

먼저 할 활동: A10, A11, A12

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다.
학생은 자신의 말로 확인 근거를 설명합니다.

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A10 · 기준 코드의 실제 실패

사례로 이해하기 · 실패 결과를 요구와 연결하는 검사

실무 상황

프로그램이 실행되었는데 검사 결과에 FAIL이 보입니다. 비전공자는 설치가 잘못됐다고 생각하기 쉽습니다. 하지만 이 검사기는 요구와 다른 출력이 나오면 의도적으로 실패 상태를 남깁니다. 실패를 읽을 수 있으면 재설치보다 수정할 규칙을 먼저 찾을 수 있습니다.

쉬운 풀이

예상은 실행 전에 규칙으로 정한 답이고 실제는 실행으로 관찰한 답입니다. 종료 코드 0은 이 검사에서 모든 사례가 맞았다는 뜻이며 1은 적어도 하나가 달랐다는 뜻입니다. 이 의미는 프로그램마다 다릅니다. TypeError가 출력에 보이면 값 종류에 맞지 않는 비교가 발생했을 수 있습니다. 누락과 종류와 범위를 먼저 확인한 뒤 경계를 비교하면 오류 상황을 정상 온도로 착각하지 않습니다.

완성 예와 풀이

압축을 푼 course에서 `python code/check_cases.py code/baseline.py`를 실행합니다. `baseline`은 28을 정상, `None`을 정상으로 반환하고 문자 비교에서는 예외가 발생합니다. 따라서 성공적인 수업 시연에서도 종료 코드가 1입니다. `reviewed.py`는 `None`을 먼저 미측정으로 처리하고 정확한 숫자 종류와 0~50 범위를 확인한 뒤 ≥ 28 을 적용합니다. `python code/check_cases.py code/reviewed.py`는 제공 아홉 사례에서 9/9입니다. 예상 목록이 같고 실행 대상만 달라졌음을 확인해야 변화의 원인을 설명할 수 있습니다.

독립 추가 복습 과제

입력 0, `None`, `True`, "28", 50, 51의 출력을 먼저 표로 작성하세요. 이후 제공 완성 함수의 조건을 위에서 아래로 따라가며 각 입력이 어느 줄에서 반환되는지 설명하세요. 조건 순서를 바꾸면 문자 비교가 어떻게 달라지는지도 판단하세요.

답과 대표 오답

0은 정상, `None`은 미측정, `True`와 "28"과 51은 입력 오류, 50은 주의입니다. "True는 1과 같으니 정상"은 오늘의 계약이 허용한 정확한 종류를 놓쳤습니다. "None은 0"은 값 없음과 유효한 영점을 혼동합니다.

확인 질문과 해설

① FAIL이면 파이썬이 실행되지 않았나요? 실행된 뒤 기대와 달랐을 수 있습니다. ② 기준만 낮추면 해결되나요? 기존 정상 경계를 깨뜨립니다. ③ 아홉 통과로 모든 입력을 보장하나요? 제공된 검사 범위만 확인한 것이므로 새로운 입력 조건은 별도로 검토합니다.

뜻과 비유. 실패 검사는 실수로 망가진 수업이 아니라 요구와 현재 동작의 차이를 드러내는 관찰입니다. 검사 프로그램은 기대와 실재를 나란히 비교합니다.

왜 필요한가. 수정 전 실패를 재현해 두면 수정 후 무엇이 달라졌는지 확인할 기준이 생깁니다.

예를 따라 읽기. `python code/check_cases.py code/baseline.py`를 실행하면 경계와 미측정 등의 차이가 FAIL로 나옵니다.

헛갈리는 반례. 종료 코드 1을 보고 무조건 재설치하지 않습니다. 이 검사기는 실패 사례가 있으면 의도적으로 1을 반환합니다.

직접 적용. 전체 검사 코드와 줄 해설 읽기 → 원본 실행 명령 확인하기 → 아홉 줄 결과에서 FAIL의 입력 찾기 → 예상과 실제 차이를 표에 남기기

설명할 질문. 테스트가 실패한 것과 실행환경이 고장 난 것은 어떻게 구별하나요?

막혔을 때. 모듈 경로 오류면 course 폴더에서 명령을 실행하고 파일 경로 인수를 빠뜨리지 않았는지 봅니다.

입력·답칸·확인 순서는 [실습지의 A10](#)와 [활동 원문](#)에 있습니다.

A10 · 따라 하기와 작성칸

A10 · 기준 코드의 실제 실패

목적

수정 전 실패를 재현해 두면 수정 후 무엇이 달라졌는지 확인할 기준이 생깁니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [check_cases.py](#), [baseline.py](#), [cases.json](#), [A10_input.md](#)

작업: [A10_notes.md](#)

실제 입력

python code/check_cases.py code/baseline.py를 실행하면 경계와 미측정 등의 차이가 FAIL로 나옵니다.

단계

1. 전체 검사 코드와 줄 해설 읽기
2. 원본 실행 명령 확인하기
3. 아홉 줄 결과에서 FAIL의 입력 찾기
4. 예상과 실제 차이를 표에 남기기

예상 결과

기준 코드의 실제 결과와 실패 원인

확인 방법

테스트가 실패한 것과 실행환경이 고장 난 것은 어떻게 구별하나요?

오류 해결

모듈 경로 오류면 course 폴더에서 명령을 실행하고 파일 경로 인수를 빠뜨리지 않았는지 봅니다.

힌트

실패 검사는 실수로 망가진 수업이 아니라 요구와 현재 동작의 차이를 드러내는 관찰입니다. 검사 프로그램은 기대와 실재를 나란히 비교합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 테스트가 실패한 것과 실행환경이 고장 난 것은 어떻게 구별하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A11 · 경계 비교 기호 수정

뜻과 비유. >는 기준보다 큰 값, >=는 기준과 같은 값까지 포함합니다. 말로 쓴 "이상"과 코드 비교 기호를 연결합니다.

왜 필요한가. 경계 한 글자는 업무 분류를 바꿉니다. 정상 값만 시험하면 이 차이를 놓칩니다.

예를 따라 읽기. 작업본의 value > threshold를 value >= threshold로 바꾸면 28의 결과가 정상에서 주의로 달라집니다.

헛갈리는 반례. 기본 threshold를 27로 내리는 것은 같은 수정이 아닙니다. 27.9까지 주의가 되어 규칙을 바꿉니다.

직접 적용. 수정 전 28결과 기록하기 → work의 비교 기호 한곳 수정하기 → 저장 후 같은 검사 다시 실행하기 → 27.9·28·28.1 변화 비교하기

설명할 질문. 기준값을 바꾸는 것과 비교 기호를 바꾸는 것은 왜 다른가요?

막혔을 때. 결과가 그대로면 저장 여부와 실행 경로를 먼저 확인합니다.

입력·답칸·확인 순서는 [실습지의 A11](#)과 [활동 원문](#)에 있습니다.

A11 · 따라 하기와 작성칸

A11 · 경계 비교 기호 수정

목적

경계 한 글자는 업무 분류를 바꿉니다. 정상 값만 시험하면 이 차이를 놓칩니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [classifier.py](#), [cases.json](#), [A11_input.md](#)

작업: [A11_notes.md](#), [classifier.py](#)

실제 입력

작업본의 value > threshold를 value >= threshold로 바꾸면 28의 결과가 정상에서 주의로 달라집니다.

단계

1. 수정 전 28결과 기록하기
2. work의 비교 기호 한곳 수정하기
3. 저장 후 같은 검사 다시 실행하기
4. 27.9·28·28.1 변화 비교하기

예상 결과

경계 수정 전후의 세 입력 결과

확인 방법

기준값을 바꾸는 것과 비교 기호를 바꾸는 것은 왜 다른가요?

오류 해결

결과가 그대로면 저장 여부와 실행 경로를 먼저 확인합니다.

힌트

는 기준보다 큰 값, >=는 기준과 같은 값까지 포함합니다. 말로 쓴 “이상”과 코드 비교 기호를 연결합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 기준값을 바꾸는 것과 비교 기호를 바꾸는 것은 왜 다른가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A12 · 누락과 잘못된 입력 처리

- 뜻과 비유.** 입력 검사는 자료가 함수의 약속에 맞는지 살펴보는 것입니다. 값이 없는 경우와 종류가 틀린 경우, 범위를 벗어난 경우를 따로 처리합니다.
- 왜 필요한가.** 없음을 0으로 채우면 실제 0 측정과 구분할 수 없고 문자에 비교 연산을 하면 실행 오류가 납니다.
- 예를 따라 읽기.** reviewed.py는 None →미측정, 숫자 아님 →입력 오류, 범위 밖 →입력 오류를 먼저 검사합니다.
- 헛갈리는 반례.** Python에서 bool은 숫자와 관련된 특성이 있지만 이 업무 계약은 온도로 받지 않으므로 type을 정확히 확인합니다.
- 직접 적용.** 완성 검사 세 부분 줄별로 읽기 → 작업본에 제공된 검사 구간 반영하기 → 0·None·True·51결과 예상하기 → 검사기로 실제 결과 확인하기
- 설명할 질문.** 값 검사를 비교보다 먼저 하는 이유는 무엇인가요?
- 막혔을 때.** 들어 쓰기가 틀리면 원본 완성 예제의 블록 범위를 대조합니다. 조건 순서도 함께 확인합니다.
- 입력·답칸·확인 순서는 [실습지의 A12](#)와 [활동 원문](#)에 있습니다.

A12 · 따라 하기와 작성칸

A12 · 누락과 잘못된 입력 처리

목적

없음을 0으로 채우면 실제 0 측정과 구분할 수 없고 문자에 비교 연산을 하면 실행 오류가 납니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [reviewed.py](#), [classifier.py](#), [cases.json](#), [A12_input.md](#)

작업: [A12_notes.md](#), [classifier.py](#)

실제 입력

reviewed.py는 None →미측정, 숫자 아님 →입력 오류, 범위 밖 →입력 오류를 먼저 검사합니다.

단계

1. 완성 검사 세 부분 줄별로 읽기
2. 작업본에 제공된 검사 구간 반영하기
3. 0·None·True·51결과 예상하기
4. 검사기로 실제 결과 확인하기

예상 결과

값 없음·종류·범위·경계가 구별된 분류기

확인 방법

값 검사를 비교보다 먼저 하는 이유는 무엇인가요?

오류 해결

틀려 쓰기가 틀리면 원본 완성 예제의 블록 범위를 대조합니다. 조건 순서도 함께 확인합니다.

힌트

입력 검사는 자료가 함수의 약속에 맞는지 살펴보는 것입니다. 값이 없는 경우와 종류가 틀린 경우, 범위를 벗어난 경우를 따로 처리합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 값 검사를 비교보다 먼저 하는 이유는 무엇인가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

5교시 · 오류 재현과 수정

15:00~15:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 오류 재현과 재요청

먼저 할 활동: A13, A14, A15

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A13 · 재현 가능한 오류 보고

사례로 이해하기 · 다른 사람이 재현할 수 있는 오류 보고

실무 상황

직원이 “AI가 이상하게 고쳤어요”라고 보고하면 동료는 어떤 입력과 파일을 봐야 할지 모릅니다. 대신 “proposal_a.py에서 27.9를 넣었더니 주의가 나왔고 요청상 정상이어야 한다”고 쓰면 바로 확인할 수 있습니다. 좋은 보고는 원인을 먼저 단정하지 않고 재현 정보를 먼저 제공합니다.

쉬운 풀이

재현은 같은 조건에서 같은 현상을 다시 보는 일입니다. 회귀는 새 변경 때문에 이전에 맞던 동작이 다시 틀리는 현상입니다. 오류 보고에는 현재 폴더, 대상 파일, 실행 명령, 입력, 기대, 실제, 관찰 방식이 들어갑니다. 추정 원인은 별도 칸에 씁니다. 학생이 강사 시연을 봤다면 개인 실행으로 적지 않고 관찰이라고 표시합니다. 제공 결과를 읽기만 했다면 기록 읽기로 구분합니다.

완성 예와 풀이

보고의 제목은 “27.9 경계 아래 값이 주의로 분류됨”입니다. 명령은 `python code/check_cases.py code/proposal_a.py`, 기대는 below 사례 정상, 실제는 주의입니다. 원인 추정은 기본 threshold가 27로 바뀐 영향입니다. 추가 요청은 “기준 28과 함수 이름, None의 미측정 계약을 유지하고 경계 비교를 검토”입니다. 수정 후에는 27.9와 28뿐 아니라 0·누락·문자·불리언·범위 밖까지 같은 검사를 다시 돌립니다. 변경한 검사가 아니라 동일한 요구 사례로 비교해야 회귀 여부를 볼 수 있습니다.

독립 추가 복습 과제

“검사에서 text 행이 TypeError”인 상황의 재현 보고 다섯 줄을 쓰세요. 확정 사실 하나와 추정 하나를 표시하고, 고친 뒤에 도 반드시 다시 확인할 다른 입력을 두 개 골라 이유를 설명하세요.

답과 대표 오답

파일과 명령, 문자 입력, 입력 오류라는 기대, TypeError라는 실제, 실행 또는 관찰 방식이 필요합니다. 종류 검사를 비교보다 먼저 하는 것이 수정 후보이며 실제 원인은 코드 확인 전에는 추정으로 둡니다. 28과 None 등을 재확인하여 경계와 누락 계약이 유지됐는지 봅니다. “전부 고쳐”는 범위를 넓혀 새 오류를 만들 수 있습니다.

확인 질문과 해설

① 통과 수만 기록하면 충분한가요? 같은 수라도 실패 입력이 다를 수 있습니다. ② 재검사에 기존 사례가 필요한 이유는 무엇인가요? 회귀를 찾습니다. ③ 원인을 확정하는 시점은 언제인가요? 코드와 재현 결과가 가설을 뒷받침할 때입니다.

뜻과 비유. 오류 보고는 실제로 한 행동·입력·기대한 결과·실제 결과를 다른 사람이 다시 따라갈 수 있게 적은 문서입니다.

왜 필요한가. “안 돼요”보다 파일 경로와 입력 하나를 제시하면 재현 범위가 좁아지고 원인 추측을 줄입니다.

예를 따라 읽기. 입력 28, 기대 주의, 실제 정상,baseline.py 실행이라는 네 정보가 있으면 경계 문제를 재현할 수 있습니다.

헛갈리는 반례. 관찰하지 않은 원인을 확정해 “AI가 고장 났다”고 쓰면 다음 확인 행동이 불명확합니다.

직접 적용. 실패한 사례 하나 고르기 → 파일·명령·입력·기대·실제 쓰기 → 추정 원인과 관찰 사실 분리하기 → 짝이 같은 문제를 재현할 수 있는 지 읽기

설명할 질문. 관찰 사실과 원인 추정은 왜 나누어 쓰나요?

막혔을 때. 실행 화면을 남기지 못 했으면 실제 출력 문구를 그대로 적고 확인하지 않은 내용은 비워 둡니다.

입력·답칸·확인 순서는 [실습지의 A13](#)와 [활동 원문](#)에 있습니다.

A13 · 따라 하기와 작성칸

A13 · 재현 가능한 오류 보고

목적

“안 돼요”보다 파일 경로와 입력 하나를 제시하면 재현 범위가 좁아지고 원인 추측을 줄입니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [review.md](#), [cases.json](#), [A13_input.md](#)

작업: [A13_notes.md](#), [review.md](#)

실제 입력

입력 28, 기대 주의, 실제 정상,baseline.py 실행이라는 네 정보가 있으면 경계 문제를 재현할 수 있습니다.

단계

1. 실패한 사례 하나 고르기

2. 파일·명령·입력·기대·실제 쓰기
3. 추정 원인과 관찰 사실 분리하기
4. 짝이 같은 문제를 재현할 수 있는 지 읽기

예상 결과

따라 할 수 있는 오류 보고 한 건

확인 방법

관찰 사실과 원인 추정은 왜 나누어 쓰나요?

오류 해결

실행 화면을 남기지 못 했으면 실제 출력 문구를 그대로 적고 확인하지 않은 내용은 비워 둡니다.

힌트

오류 보고는 실제로 한 행동·입력·기대한 결과·실제 결과를 다른 사람이 다시 따라갈 수 있게 적은 문서입니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 관찰 사실과 원인 추정은 왜 나누어 쓰나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A14 · 수정 요청을 좁히기

뜻과 비유. 추가 요청은 실패한 조건과 유지할 조건을 함께 적는 것입니다. 문제가 하나 발견됐다고 전체 프로그램을 다시 만드는 요청으로 넓히지 않습니다.

왜 필요한가. 작은 수정은 이미 통과한 사례를 보존하는지 재확인하기 쉽습니다.

예를 따라 읽기. “28의 기대는 주의인데 정상입니다. 기준 28과 함수 이름을 유지하고 비교 조건만 검토해 주세요.”

햇갈리는 반례. “전부 새로 고쳐”는 빈 값 처리나 이름까지 바꾸어 새 문제를 만들 수 있습니다.

직접 적용. 이전 실패에서 추가 요청한 문단 쓰기 → 수정 대상과 유지 대상 분리하기 → 응답 B가 주장한 범위 읽기 → 실행으로 확인할 주장 표시하기

설명할 질문. 추가 요청에서도 유지할 조건을 다시 적는 이유는 무엇인가요?

막혔을 때. 응답에 없는 검증을 했다고 추측하지 말고 별도 확인 항목으로 남깁니다.

입력·답칸·확인 순서는 [실습지의 A14](#)와 [활동 원문](#)에 있습니다.

A14 · 따라 하기와 작성칸

A14 · 수정 요청을 좁히기

목적

작은 수정은 이미 통과한 사례를 보존하는지 재확인하기 쉽습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [agent_response_b.md](#), [task.md](#), [review.md](#), [A14_input.md](#)

작업: [A14_notes.md](#), [task.md](#), [review.md](#)

실제 입력

“28의 기대는 주의인데 정상입니다. 기준 28과 함수 이름을 유지하고 비교 조건만 검토해 주세요.”

단계

1. 이전 실패에서 추가 요청한 문단 쓰기
2. 수정 대상과 유지 대상 분리하기
3. 응답 B가 주장한 범위 읽기
4. 실행으로 확인할 주장 표시하기

예상 결과

증상·기대·수정 범위가 좁혀진 재 요청

확인 방법

추가 요청에서도 유지할 조건을 다시 적는 이유는 무엇인가요?

오류 해결

응답에 없는 검증을 했다고 추측하지 말고 별도 확인 항목으로 남깁니다.

힌트

추가 요청은 실패한 조건과 유지할 조건을 함께 적는 것입니다. 문제가 하나 발견됐다고 전체 프로그램을 다시 만드는 요청으로 넓히지 않습니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 추가 요청에서도 유지할 조건을 다시 적는 이유는 무엇인가요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
실제 결과·근거: _____
다음에 확인할 것: _____

A15 · 회귀 확인과 증거

뜻과 비유. 회귀는 수정한 뒤 예전에 맞던 동작이 다시 틀리는 현상입니다. 수정한 사례 뿐 아니라 관련된 기존 사례도 함께 확인합니다.

왜 필요한가. 경계를 고치며 기준을 27로 바꾸면 28 은 맞아도 27.9가 틀려집니다.

예를 따라 읽기. baseline-proposal_a-reviewed의 아홉 결과를 같은 기대 값으로 비교합니다.

헛갈리는 반례. 검사를 새 코드에 유리하게 바꾸거나 통과 한 줄만 보고하면 근거가 사라집니다.

직접 적용. 세 후보에 같은 검사 실행하기 → 통과 수와 실패 입력 표로 비교하기 → 요구 위반과 실패의 연결 쓰기 → 원본 검사 파일이 유지됐는지 확인하기

설명할 질문. 통과 수만 같으면 같은 품질이라고 할 수 있나요?

막혔을 때. FAIL 수가 줄어도 새로운 실패가 입력되었는지 각 행을 확인합니다.

입력·답칸·확인 순서는 [실습지의 A15](#)와 [활동 원문](#)에 있습니다.

A15 · 따라 하기와 작성칸

A15 · 회귀 확인과 증거

목적

경계를 고치며 기준을 27로 바꾸면 28 은 맞아도 27.9가 틀려집니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [baseline.py](#), [proposal_a.py](#), [reviewed.py](#), [check_cases.py](#), [A15_input.md](#)

작업: [A15_notes.md](#)

실제 입력

baseline-proposal_a-reviewed의 아홉 결과를 같은 기대 값으로 비교합니다.

단계

- 1. 세 후보에 같은 검사 실행하기
- 2. 통과 수와 실패 입력 표로 비교하기
- 3. 요구 위반과 실패의 연결 쓰기
- 4. 원본 검사 파일이 유지됐는지 확인하기

예상 결과

세 후보의 비교표와 선택 근거

확인 방법

통과 수만 같으면 같은 품질이라고 할 수 있나요?

오류 해결

FAIL 수가 줄어도 새로운 실패가 입력되었는지 각 행을 확인합니다.

힌트

회귀는 수정한 뒤 예전에 맞던 동작이 다시 틀리는 현상입니다. 수정한 사례 뿐 아니라 관련된 기존 사례도 함께 확인합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 통과 수만 같으면 같은 품질이라고 할 수 있나요? → _____

- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
실제 결과·근거: _____
다음에 확인할 것: _____

6교시 · 새 요청과 인수인계

16:00~16:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 새 기준과 인계

먼저 할 활동: A16, A17, A18

이 교시는 50분 운영이며 아래 세 개념을 연결합니다. 필수 경로를 먼저 진행하고 추가 복습은 수업 진행에 따라 배정합니다. 학생은 자신의 말로 확인 근거를 설명합니다.

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A16 · 새 조건의 영향 분석

사례로 이해하기 · 새 기준과 완료 범위를 인계하기

실무 상황

담당자가 새 장소에는 기준 30을 적용해 달라고 승인했습니다. 기존 모든 장소의 기본값을 30으로 바꾸면 이전 업무가 달라 집니다. 새 요청이 특정 장소만의 조건인지 전체 기본값 변경인지 먼저 확인하고 영향 받는 호출과 검사를 찾습니다.

쉬운 풀이

함수 인수는 호출할 때 전달하는 입력입니다. 기본값은 그 인수를 생략했을 때 사용하는 값입니다. `classify(29.9, threshold=30)`과 `classify(29.9)`는 같은 입력 온도지만 기준이 다르므로 결과가 다를 수 있습니다. 요구 변경은 검사의 기대를 바꿀 수 있으나 어떤 승인된 요구에 따른 변경인지 근거를 남겨야 합니다. 기존 실패를 숨기려고 기대를 바꾼 상황과 구별합니다.

완성 예와 풀이

새 장소 요청서에 기준 30, 기본 계약 28 유지, 유효 범위 0~50 유지라고 씁니다. 새 호출의 29.9는 정상, 30과 30.1은 주의입니다. 기본 호출의 28은 여전히 주의입니다. None은 두 호출 모두 미측정이고 범위 밖은 입력 오류입니다. 인계에는 수정한 파일, 변경 조건, 기본 아홉 사례 결과, 새 경계 세 사례 결과, 실제 AI 호출 여부를 씁니다. 오늘은 제공 응답을 검토한 오프라인 활동이며 실제 센서 안전 판단과 실제 AI 서비스 연결은 확인하지 않았다고 기록합니다.

독립 추가 복습 과제

기준 30을 특정 장소에만 적용하는 요청과 전체 기본값을 30으로 바꾸는 요청을 각각 한 문장으로 작성하세요. 두 요청에서 기대가 달라질 기본 호출 한 개를 고르고, 동료가 다시 확인할 명령과 자료 위치를 인계문에 넣으세요.

답과 대표 오답

특정 장소에는 threshold 인수를 명시하고 전체 변경에는 기본값 변경 승인을 명시합니다. 기본 classify(28)은 전자에서 주의, 후자에서 정상입니다. “새 검사가 통과했으니 모든 서비스 운영 완료”는 검증 범위를 넘었습니다. “새 요청은 절대 기대를 바꾸면 안 된다”도 승인된 요구 변경을 무시합니다.

확인 질문과 해설

① 기본값과 인수는 왜 구별하나요? 영향 받는 호출 범위가 다릅니다. ② 동료가 같은 결과를 보려면 무엇이 필요한가요? 대상 파일·실행 위치·입력·기대를 연결한 기록입니다. ③ 아직 모르는 사항은 어떻게 전달하나요? 실제 환경, 장비, 계정 등 미확인 항목과 다음 확인 담당을 구체적으로 남깁니다.

뜻과 비유. 새 업무 조건은 기존 규칙과 독립인지 먼저 확인한 뒤 작은 사례로 설계합니다. 오늘은 추가 코드보다 변경이 영향 줄 입력을 설명하는 일에 집중합니다.

왜 필요한가. 특수 기준을 추가하면 조건 순서와 기존 계약을 함께 검토해야합니다.

예를 따라 읽기. 담당자가 기준을 30으로 바꿔 달라고 새로 요청했다면 29.9·30·30.1을 새 경계로 예상합니다.

헛갈리는 반례. 승인된 새 요청 없이 기준을 바꾸는 것과 업무 담당자가 조건을 변경한 것은 다릅니다.

직접 적용. 기준 30이라는 새 요청서 작성하기 → 영향 받을 기대 값 찾기 → 29.9·30·30.1의 예상 작성하기 → 기본 기준 28 을 유지할지 인수로 전달할지 설명하기

설명할 질문. 새 조건에서 어떤 기존 검사를 그대로 써야하나요?

막혔을 때. 기본 기준 변경과 함수 인수 전달을 혼동하면 classify(30,threshold =30)호출 예를 읽습니다.

입력·답칸·확인 순서는 [실습지의 A16](#)와 [활동 원문](#)에 있습니다.

A16 · 따라 하기와 작성칸

A16 · 새 조건의 영향 분석

목적

특수 기준을 추가하면 조건 순서와 기존 계약을 함께 검토해야합니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [reviewed.py](#), [task.md](#), [A16_input.md](#)

작업: [A16_notes.md](#), [task.md](#)

실제 입력

담당자가 기준을 30으로 바꿔 달라고 새로 요청했다면 29.9·30·30.1을 새 경계로 예상합니다.

단계

1. 기준 30이라는 새 요청서 작성하기
2. 영향 받을 기대 값 찾기
3. 29.9·30·30.1의 예상 작성하기
4. 기본 기준 28 을 유지할지 인수로 전달할지 설명하기

예상 결과

추가 조건의 영향 표와 검증 계획

확인 방법

새 조건에서 어떤 기준 검사를 그대로 써야하나요?

오류 해결

기본 기준 변경과 함수 인수 전달을 혼동하면 `classify(30,threshold =30)`호출 예를 읽습니다.

힌트

새 업무 조건은 기존 규칙과 독립인지 먼저 확인한 뒤 작은 사례로 설계합니다. 오늘은 추가 코드보다 변경이 영향 줄 입력을 설명하는 일에 집중합니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 새 조건에서 어떤 기준 검사를 그대로 써야하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____

- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A17 · 동료의 변경을 설명하기

- 뜻과 비유.** 동료 검토는 작성자의 설명 없이도 요구·변경·검사 근거를 찾을 수 있는지 확인하는 과정입니다.
- 왜 필요한가.** 다른 사람에게 설명할 수 없다면 어떤 줄을 왜 바꿨는지 자신도 놓친 부분이 있을 수 있습니다.
- 예를 따라 읽기.** 짝은 None 처리와 >=조건을 찾고 요청서 및 검사 입력을 한 곳씩 연결합니다.
- 햇갈리는 반례.** “테스트됐음”한 줄만 있으면 실행한 대상과 입력이 불명확합니다.
- 직접 적용.** 작업본과 검토 노트를 짝에게 전달하기 → 요구 두 개에 대응하는 줄 찾기 → 검사 결과 한 개를 재현하기 → 부족한 설명하나 보완하기
- 설명할 질문.** 동료가 다시 확인하려면 어떤 정보가 있어야하나요?
- 막혔을 때.** 짝의 작업본을 바로 덮어쓰지 말고 수정 제안을 먼저 기록합니다.
- 입력·답칸·확인 순서는 [실습지의 A17](#)와 [활동 원문](#)에 있습니다.

A17 · 따라 하기와 작성칸

A17 · 동료의 변경을 설명하기

목적

다른 사람에게 설명할 수 없다면 어떤 줄을 왜 바꿨는지 자신도 놓친 부분이 있을 수 있습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [classifier.py](#), [review.md](#), [request.md](#), [A17_input.md](#)

작업: [A17_notes.md](#), [classifier.py](#), [review.md](#)

실제 입력

짝은 None 처리와 >=조건을 찾고 요청서 및 검사 입력을 한 곳씩 연결합니다.

단계

- 1. 작업본과 검토 노트를 짝에게 전달하기
- 2. 요구 두 개에 대응하는 줄 찾기
- 3. 검사 결과 한 개를 재현하기
- 4. 부족한 설명하나 보완하기

예상 결과

요구·줄·결과를 이어 설명한 동료 검토

확인 방법

동료가 다시 확인하려면 어떤 정보가 있어야하나요?

오류 해결

짝의 작업본을 바로 덮어쓰지 말고 수정 제안을 먼저 기록합니다.

힌트

동료 검토는 작성자의 설명 없이도 요구·변경·검사 근거를 찾을 수 있는지 확인하는 과정입니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 동료가 다시 확인하려면 어떤 정보가 있어야하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A18 · 완료 범위와 다음 확인

뜻과 비유. 인수인계는 요청한 일, 실제 바뀐 것, 확인한 증거, 남은 조건을 다음 사람이 이해할 수 있게 남기는 일입니다.

왜 필요한가. 합성 데이터 검사와 실제 센서 운영은 다른 범위입니다. 확인하지 않은 서비스 호출이나 장비 연결을 완료라고 쓰지 않습니다.

예를 따라 읽기. 분류기 아홉 사례 통과·오프라인 응답 검토·실제 AI 호출 미 수행을 함께 적습니다.

헛갈리는 반례. AI가 코드를 썼다고 가정하거나 검사 통과만으로 실제 안전 시스템 완성이라고 표현하지 않습니다.

직접 적용. 세 문장 인계 작성하기 → 실제 AI 호출 여부 명시하기 → 미검증 조건 두 개 쓰기 → 새 질문:0과 None·기준과 경계·설명과 증거 답하기

설명할 질문. 자료 작성과 실제 운영 검증을 어떻게 구분하나요?

막혔을 때. 완료 문장이 추상적이면 파일·입력 수·실행 방법을 넣습니다.

입력·답칸·확인 순서는 [실습지의 A18](#)와 [활동 원문](#)에 있습니다.

공식 근거와 원본

확인일2026-09-26. 정의는 짧게 요약하고 활동·데이터·설명은 수업용으로 새로 작성했습니다.

- 원본 Track A / A3.pptx, 슬라이드 119~163: 기존 요청·에이전트 흐름 참고. 현행 도구 사용법으로 단정하지 않습니다. 원본은 강사용 로컬 자료에 보존합니다.

- [코드에이전트의 작업 흐름](#): 읽기·변경·실행·검토의 개념 참고, 특정 계정 사용 실습 아님

- [Python 조건과 함수](#): 함수·조건·반환 기본 동작

개념 보충 공식 근거 (2026-10-05 확인)

- [GitHub 코드에이전트의 도구 작업과 검토](#)

A18 · 따라 하기와 작성칸

A18 · 완료 범위와 다음 확인

목적

합성 데이터 검사와 실제 센서 운영은 다른 범위입니다. 확인하지 않은 서비스 호출이나 장비 연결을 완료라고 쓰지 않습니다.

준비

입력 카드와 지정 파일을 나란히 엽니다. 원본을 보존하고 work의 답칸과 작업 복사본을 사용합니다. 코드 전체와 모든 줄 해설을 읽고 예상 결과를 먼저 씁니다.

입력: [review.md](#), [A18_input.md](#)

작업: [A18_notes.md](#), [review.md](#)

실제 입력

분류기 아홉 사례 통과·오프라인 응답 검토·실제 AI 호출 미 수행을 함께 적습니다.

단계

- 1. 세 문장 인계 작성하기
- 2. 실제 AI 호출 여부 명시하기
- 3. 미검증 조건 두 개 쓰기
- 4. 새 질문:0과 None-기준과 경계·설명과 증거 답하기

예상 결과

완료·증거·미확인 범위가 있는 인계

확인 방법

자료 작성과 실제 운영 검증을 어떻게 구분하나요?

오류 해결

완료 문장이 추상적이면 파일·입력 수·실행 방법을 넣습니다.

힌트

인수인계는 요청한 일, 실제 바뀐 것, 확인한 증거, 남은 조건을 다음 사람이 이해할 수 있게 남기는 일입니다.

빠른 학생의 확장

필수 확인을 마친 뒤 같은 규칙의 다른 입력 하나를 만들고 예상과 실제 결과를 비교합니다. 새 기능 전체를 독립 작성하는 것은 선택입니다.

학생 답칸

- 개념 설명: 자료 작성과 실제 운영 검증을 어떻게 구분하나요? → _____
- 입력 파일과 내가 바꾼 위치: _____
- 실행 전 예상 결과: _____
- 실제 관찰 결과와 확인 방법: _____
- 예상과 달랐던 점·복구·남은 질문: _____

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

용어와 코드 기호 찾아보기

AI 코드에이전트 쉬운 용어

용어를 말한 뒤 예제의 실제 위치를 함께 짚습니다.

용어	뜻
코드에이전트	요청을 해석하고 코드 파일·실행 도구를 이용해 작업을 이어 가는 AI 도구
프롬프트	목표·자료·범위·확인 방법을 전달하는 요청
컨텍스트	이번 일을 이해하는 데 필요한 관련 설명과 자료
입출력 계약	받을 값과 돌려줄 결과에 대한 약속
diff	바뀌기 전후의 파일과 줄을 보여 주는 비교
경계값	판단 규칙이 바뀌는 기준 주변의 값
회귀	수정 뒤 예전에 맞던 동작이 틀리는 현상
재현	같은 입력과 조건으로 같은 현상을 다시 확인하기
검증	미리 정한 기대와 실제 결과를 대조하기
미측정	아직 측정값이 없는 상태로 실제0과 다름

오늘 배운 내용 확인하기

2026-10-20 확인·평가

공식 수료 기준이 아닌 형성평가입니다. 개인 설치 성공만으로 개념 이해를 판단하지 않습니다. 학생은 직접 수정·예상·대조·설명을 수행합니다.

구간	확인 질문	총족 근거
작업을 맡기는 방법	자료가 많으면 항상 더 좋은 요청이 되나요?	최소 관련 자료 목록과 선택 이유
근거가 있는 요청서	좋은 요청에 실행 전 기대 결과가 들어가는 이유는 무엇인가요?	입력·제약·검사 가능한 작업 요청
변경된 코드 읽기	파일 이름이 같은데 다른 결과가 나올 수 있는 이유는 무엇인가요?	원본·작업본·실행 경로와 복구 기준
예상과 실제 결과	값 검사를 비교보다 먼저 하는 이유는 무엇인가요?	값 없음·종류·범위·경계가 구별된 분류기
오류 재현과 수정	통과 수만 같으면 같은 품질이라고 할 수 있나요?	세 후보의 비교표와 선택 근거
새 요청과 인수인계	자료 작성과 실제 운영 검증을 어떻게 구분하나요?	완료·증거·미확인 범위가 있는 인계

최종 제출

- work의18활동 답칸과 직접 수정한 예제 복사본
- 바꾼 파일·입력·예상·실제 결과를 연결한 검수 기록
- 완료 범위·실제 실행 방식·아직 확인하지 않은 조건의 인수인계

판단 방법

스스로 설명함 / 예를 보고 설명함 / 다시 연습 필요를 각 구간에 기록합니다. 결과가 틀렸더라도 근거를 찾아 수정하고 다시 확인한 과정을 남깁니다. 관찰하지 않은 출력을 실제 실행으로 쓰지 않습니다. 빠른 학생의 확장은 필수 기준에 포함하지 않습니다.

개념 보충 전이 문제 · 독립 추가 복습

교시별 필수 설명을 다른 상황에 적용하는 추가 복습입니다. 50분 필수 경로에 모두 포함되어 완수된 것으로 처리하지 않습니다. 답안은 먼저 학생이 작성하고 이후 아래 해설을 대조합니다. 설치·계정 성공 여부와 판단 근거를 분리합니다.

전이 1 · 작업 요청과 공유 자료

문제: 온도 경계 오류를 맡길 때 고객 연락처, 실제 API 키, 현재 함수, 가상 검사 입력, 실패 로그 중 무엇을 제공하나요? 다섯 줄 요청을 작성하세요.

해설·확인 기준: 현재 함수·가상 검사 입력·비밀을 제거한 실패 로그를 제공합니다. 목적·대상·유지 조건·허용 행동·검증을 씁니다. 실제 키와 고객 연락처는 이번 문제의 근거에 필요하지 않습니다. 자료를 많이 보내는 것이 목표라는 답은 부족합니다.

전이 2 · 새 경계 규칙

문제: 업무 규칙은 5 초과 점검입니다. 4.9·5·5.1과 None의 기대를 쓰고 28 이상 주의 규칙과 비교 기호 차이를 설명하세요.

해설·확인 기준: 정상·정상·점검이며 누락은 별도 미측정입니다. 초과는 >, 이상은 >=입니다. 함수 출력 문자열은 새로운 업무 계약에 맞게 명시합니다. 기준과 기호를 함께 설명하면 충족입니다.

전이 3 · 주장과 근거

문제: 응답은 "수정·검사·실제 운영 모두 완료"라고 합니다. 제공 근거는 코드 diff와 로컬 가상 검사 출력뿐입니다. 보고할 수 있는 범위와 남은 질문을 쓰세요.

해설·확인 기준: 실제 변경과 해당 로컬 검사 결과까지 보고하고 실제 장비·서비스 운영은 미확인으로 남깁니다. 대상 파일·명령·입력이 필요합니다. 증거 부족을 자동 실패라고 단정하지 않습니다.

전이 4 · 잘못된 입력의 해석

문제: 0·None·True·문자 "28"·50·51을 현재 분류 계약으로 판단하고 종류 검사를 비교보다 먼저 해야 하는 이유를 쓰세요.

해설·확인 기준: 정상·미측정·입력 오류·입력 오류·주의·입력 오류입니다. 문자나 불리언은 허용한 정확한 숫자 종류가 아니며 문자 비교는 예외를 낼 수 있습니다. None을 0으로 바꾸면 측정과 누락을 합칩니다.

전이 5 · 재현 가능한 보고

문제: 입력 문자 "뜨거움"에서 TypeError가 관찰되었습니다. 다른 사람이 재현할 수 있는 보고와 수정 뒤 회귀 확인 두 사례를 쓰세요.

해설·확인 기준: 파일·현재 위치·명령·입력·기대 입력 오류·실제 TypeError·관찰 방식을 씁니다. 종류 검사 누락은 코드 확인 전 가설입니다. 기존 28 경계와 None 등을 다시 확인하여 이전 계약을 보존합니다.

전이 6 · 기본값과 특정 호출

문제: 특정 장소만 기준 30입니다. classify(28)과 classify(29.9,threshold=30)의 기대를 쓰고 인계에 남길 미확인 범위 두 개를 제시하세요.

해설·확인 기준: 기본 호출은 주의, 기준 30 호출은 정상입니다. 기본값 28을 유지하고 특정 호출 인수를 지정합니다. 실제 AI 서비스 연결과 실제 센서 운영은 미확인입니다. 기본 전체 변경은 별도 요구라는 점을 설명하면 충족입니다.

파일과 다음 수업

링크를 눌러 파일을 내려받습니다. 예제 ZIP은 압축을 풀고 README부터 읽습니다. 원본 code·data는 보존하고 work의 작업본을 수정합니다. 파일이 저장된 폴더와 확장명을 확인하세요.

실습 예제 코드와 데이터

AI 코드에이전트 PPT

처음 시작 안내

용어와 코드 기호

학생용 교안

학생 실습 안내

확인 문제

학생 교안 PDF · 내용 보강본

설치·로그인·장비 준비와 학습 성취는 구분합니다. 이 교안의 배정 시간은 실제 초심자 리허설 결과가 아닙니다. 코드·제공 출력·실물 검증의 한계는 해당 웹 수업과 교안 설명을 따릅니다.